

Zuschnitt auf die valuvita GmbH. Im Auftrag der valuvita GmbH wird ausschließlich die Produktkategorie **Lebenswerk** eingesetzt: Die erzählende Person berichtet ihr eigenes Leben und erhält dafür einen eigenen Zugang. Die Anwendung beherrscht weitere Anlässe — Gedenkbuch, Geburtstag, Jubiläum, Abschied, Geburt, Mutmachbuch, Anamnesebogen —, die hier **nicht beauftragt** und deshalb aus dieser Fassung entfernt sind. Maßgeblich für das Auftragsverhältnis ist diese Fassung.

Sicherheit & technische Maßnahmen (TOM)

Kurzdokumentation der technischen Schutzmaßnahmen der Lebensgeschichten-App (DSGVO Art. 32).
Stand: 2026-08-03. Produktion: lebensgeschichten.ai.

1. Verschlüsselung

In Transit (Übertragung)

- Die Anwendung wird ausschließlich über HTTPS/TLS ausgeliefert; der Betrieb läuft auf Azure Container Apps, das eingehende Verbindungen auf HTTPS zwingt und das Zertifikat für `lebensgeschichten.ai` verwaltet.
- Die Datenbankverbindung ist TLS-pflichtig (`sslmode=require`).
- Alle Backend-Aufrufe an Drittanbieter laufen über HTTPS:
 - Microsoft Azure OpenAI (LLM, EU): `https://<resource>.services.ai.azure.com` (gpt-4.1, DataZone EU/westeurope)
 - Microsoft Azure AI Speech (TTS/STT, EU): `https://<region>.tts.speech.microsoft.com` bzw. `https://<region>.api.cognitive.microsoft.com`
 - Microsoft Azure Foundry – FLUX.2 [pro] (Bilderzeugung, EU): `https://<resource>.services.ai.azure.com`
 - **Keine US-Pfade.** Ist ein Azure-Dienst nicht erreichbar, meldet der Endpunkt einen Fehler, statt auf einen Anbieter außerhalb der EU auszuweichen.
- Keine unverschlüsselten (`http://`) Produktivverbindungen im Code.

At Rest (Speicherung)

- Azure Database for PostgreSQL Flexible Server (North Europe) und Azure Blob Storage: serverseitig AES-256-verschlüsselt (Azure-Standard).
- Der Container für die Buchbilder und hochgeladenen Fotos ist **privat**; Lesezugriffe laufen ausschließlich über kurzlebige signierte Links (SAS, 1 Stunde).
- Secrets der Container-App: verschlüsselt gespeichert.

2. Secret-Management

- Secrets liegen als Container-App-Secrets in Azure und werden von den Umgebungsvariablen nur referenziert; lokal in einer nicht versionierten `.env`.

- `.gitignore` schließt `.env` / `.env.*` aus; eine `.env` wurde nie committet (Git-History geprüft).
- **Das Frontend referenziert keine einzige Umgebungsvariable** — alle Geheimnisse bleiben serverseitig. Der Browser spricht nur die eigene API an, nie direkt Azure. Das gilt auch für das Live-Sprachgespräch (Abschnitt 5).
- Die Datenbank hat **keinen öffentlichen Endpunkt** und ist nur aus dem Backend mit einem eigenen Datenbankbenutzer erreichbar.
- Pflicht-Secrets siehe `CLAUDE.md` → „Required environment variables“.

Rotation (organisatorisch): Bei Verdacht oder Personalwechsel die Keys beim jeweiligen Anbieter neu erzeugen und in der Container-App ersetzen; `ADMIN_TOKEN_SECRET` -Wechsel invalidiert alle laufenden Admin-Sessions.

3. Zugriffs-/Audit-Logging

- Dauerhaftes Audit-Log in der Datenbank — die Laufzeit-Protokolle des Containers sind flüchtig und als Nachweis ungeeignet.
- Geschrieben über eine eigene Protokollfunktion (fail-open: ein Logging-Fehler bricht nie die eigentliche Aktion ab). **PII-arm:** nur Akteur (uid), Aktion, Ziel-Code/-ID, IP, Zeitstempel — keine Inhalte, kein Passwort-Material.
- Protokollierte Aktionen: erfolgreiche und fehlgeschlagene Anmeldung; Anlegen, Ändern und Löschen eines Buchprojekts; Löschen eines Beitrags; Anlegen, Ändern und Löschen eines Benutzerkontos.
- **Auswertung:** im Dashboard (Benutzerverwaltung).
- **Aufbewahrung:** Einträge älter als 365 Tage werden vom täglichen Cron-Lauf automatisch entfernt.

4. Zugriffskontrolle (Kurzüberblick)

- Admin-Endpunkte: HMAC-signierter Bearer-Token mit 12 h Ablauf, keine unsicheren Defaults.
- Mehrbenutzer-Isolation: Nicht-Admins sehen/bearbeiten nur eigene Buchprojekte. Fremde Projekte werden nicht als „gesperrt“, sondern als „nicht vorhanden“ beantwortet, damit sich Zugangscodes nicht durchprobieren lassen.
- Öffentlicher Beitragenden-Flow: ein Beitrag nur per geheimer Beitrags-ID (Capability); der öffentliche Buch-Endpunkt liefert nur eine Feld-Allowlist.
- Offene KI-Proxies an gültigen Code gebunden + Rate-Limiting, Login zusätzlich Brute-Force-gebremst.
- **Keine Row Level Security.** Sie wäre hier wirkungslos: Die Datenbank ist nicht über eine öffentliche Datenschnittstelle erreichbar, sondern ausschließlich aus dem Backend mit einem eigenen Datenbankbenutzer. Der Zugriffsschutz liegt damit in der Anwendung (Abschnitte oben) und im Netzzugang, nicht in Tabellenrichtlinien.

5. Live-Sprachgespräch (Azure Voice Live)

Der optionale vierte Mikrofon-Modus hält während des Interviews eine durchgehende Audioverbindung. Weil dabei Art.-9-Daten als Rohaudio fließen, gelten eigene Maßnahmen:

- **Kein Direktkontakt Browser ↔ Azure.** Ein WebSocket-Relay im eigenen Backend steht dazwischen. Grund: Ein Browser-WebSocket kann keinen `Authorization`-Header setzen — Microsofts Beispiele hängen den Ressourcenschlüssel als Query-Parameter an, er läge damit im Browser. Über das Relay bleibt der Schlüssel serverseitig.
- **Region fest verdrahtet.** Die Sitzung geht ausschließlich an die Ressource in **Sweden Central**. Der von Microsoft für Browser empfohlene WebRTC-Pfad (`/voice-live/realtime/calls`) wird bewusst **nicht** benutzt: Er nutzt laut Doku „global standard deployments“ und routet zur nächstgelegenen Region.
- **Modell-Allowlist.** Die EU-Residenz hängt am Deployment-Typ des Chat-Modells, und den gibt Microsoft je Region und Modell vor. Ein nicht freigegebenes Modell (z. B. `gpt-realtime` , das nur als „Global standard“ existiert) **deaktiviert den Dienst**, statt ihn still global laufen zu lassen. Eine Fehlkonfiguration der Env-Variable kann die Rechtsgrundlage damit nicht unbemerkt aushebeln.
- **Nachrichten-Allowlist zum Dienst hin** (`CLIENT_ALLOWED_TYPES`). Der Browser darf Audio und Antwortanforderungen schicken, aber **kein** `session.update` — Modell, Stimme, Sprache und Transkription setzt allein das Relay. Sonst könnte ein manipulierter Client die Sitzung auf ein teureres oder außereuropäisches Modell umbiegen.
- **Kurzlebiges Ticket statt Dauerzugang.** `POST /api/voicelive-token` prüft Buch-Code, Budget und Sprache und stellt ein HMAC-signiertes Ticket mit 2 Minuten Gültigkeit aus (`ADMIN_TOKEN_SECRET`). Es dient nur dem Verbindungsaufbau.
- **Nie Standard, immer bewusste Wahl.** Der Modus steht allen offen, ist aber **niemals voreingestellt**: Voreinstellung bleibt die Mischform (Mikrofon öffnet automatisch, die erzählende Person beendet per Knopfdruck). Der Live-Modus wird ausschließlich aktiv, wenn die Person ihn im Menü „Mikrofon-Modus“ selbst auswählt, und lässt sich jederzeit wieder verlassen. Jeder Fehler fällt still auf die Mikrofon-Modi zurück.
- **Im Begleiteten Modus abgeschaltet.** Sprechen zwei Menschen abwechselnd (Begleitperson), bleibt der Live-Modus aus — eine durchgehende Verbindung mit Ausrichtung auf eine Stimme passt dort nicht.
- **Kein Audio-Mitschnitt.** Gespeichert wird ausschließlich das Transkript in derselben Struktur wie im Mikrofon-Modus. Voice Live selbst speichert nach Microsoft-Doku nichts; die optionale Protokollierung für Support-Fälle ist nicht aktiviert.
- **Kostendeckel greift auch hier.** Jede Antwortrunde wird über `costRealtime` auf das Buch gebucht; die Budget-Obergrenze stoppt die Sitzung wie jede andere KI-Funktion.